

AiMn KISS V3 Strategy - Holy Bible

ONE STRATEGY, ONE ENGINE, DIFFERENT MEMORIES

AiMn KISS V3 Strategy — ONE STRATEGY, ONE ENGINE, DIFFERENT MEMORIES

CORE RULE

ONE question: Has market changed direction?

Three states: LONG (up), SHORT (down), FLAT (sideways)

Transitions that ENTER: SHORT→LONG, LONG→SHORT, FLAT→LONG, FLAT→SHORT

Transitions that WAIT: LONG→FLAT, SHORT→FLAT = wait for next

V-Long = falling → rising = ideal LONG

V-Short = rising → falling = ideal SHORT

Shape is NOT strategy, transition IS.

WHY OLD METHOD FAILED

for i in 124..2016 forward:

if rsi<=25 → trade

Result: 20 trades, 0.007% avg → commission killer

NEW METHOD - WHAT? WHEN?

Remember WHAT=trend, WHEN=time. Check ONLY new candles since last memory.

One brain, separate memories: live, backtest, tuner each own notebook.

START FROM NEW CANDLE GOING BACK to find last transition, not forward scan.

IMPLEMENTATION

Engine: engine/trend_engine.py

```
class TrendMemory:
    trend: str # LONG/SHORT/FLAT
    time: datetime
    entry_price: float
    peak_price: float
    def update(new_trend, time, price): ...

    def get_market_state(closes, idx, window=20) -> LONG/SHORT/FLAT:
```

simple slope vs MA

```
ma = sum(closes[idx-window:idx])/window
if closes[idx] > ma*1.002: return LONG
if closes[idx] < ma*0.998: return SHORT
return FLAT

def confirm_transition(closes, from_idx, new_trend, confirmBars=3) -> bool:
```

next 2-4 candles must stay in new_trend, ignore 1-bar noise

```
count=0
for j in range(1, confirmBars+1):
    if get_market_state(closes, from_idx+j) == new_trend:
        count+=1
return count >=2

def is_v_shape(lows, highs, closes, idx) -> V-LONG/V-SHORT/None:
```

V-Long: low at idx-1, higher before and after

V-Short: high at idx-1

...

```
def find_last_transition(closes, highs, lows, from_idx=None) -> {from,to,at,v}:
```

START FROM NEW CANDLE GOING BACK

...

```
def detect_transitions(closes, highs, lows, window, confirm, trail_pct) -> [pnl]:
```

core backtest loop using memory

...

```
def emergency_rsi_exit(closes, idx, direction) -> bool:
```

```
if direction==LONG and rsi<20: return True
```

```
if direction==SHORT and rsi>80: return True
```

```
return False
```

```
def check_stop_loss(entry, current, direction, stop_pct) -> bool:
```

...

Exit Rules

Follow trend, trailing 1.5% from peak, don't exit on noise, exit on meaningful transition.

RSI = emergency only. Stop = final.

Commission Reality

QQQ SHORT 15 trades 73% win 1.95% total 0.13% avg = WINNER

NVDA SHORT 13 trades 46% 2.58% 0.19%

AAPL LONG 6 trades 50% 2.52% 0.42%

MSFT LONG 14 trades 50% 1.14% 0.08%

Losers: SPY LONG -0.26%, QQQ LONG -3.23% 12.5% → regime, use opposite.

Flask Routes

/api/trend/state -> get_market_state

/api/trend/transitions -> detect_transitions

/api/run_tuning -> uses same engine

JS: static/trend_engine.js same functions for frontend.

File Structure

aimn-trade-final/

doc/strategy/AiMn-KISS-Strategy-V3.md

doc/strategy/AiMn-KISS-Strategy-V3-full.md

engine/trend_engine.py <- SHARED ENGINE
engine/tuning/auto_tuner.py -> uses trend_engine
app.py -> /docs/strategy route

Dashboard Button

■ Strategy V3